



Linux for Beginners

A Practical Guide

Chapter 2 of 12

Navigating the Terminal: Your First Commands

Contents

2.1	The Shell: What It Is and How It Thinks
2.2	Anatomy of a Command
2.3	Where Are You? — pwd
2.4	What's Here? — ls
2.5	Moving Around — cd
2.6	Getting Help: man, --help, and whatis
2.7	Command History and Shortcuts
2.8	Tab Completion: Your Best Friend
2.9	Clearing the Screen and Exiting
2.10	Essential First Commands: Quick Reference
2.11	Chapter Summary

Chapter 2

Navigating the Terminal: Your First Commands

2.1 The Shell: What It Is and How It Thinks

When you open a terminal, you're not talking directly to the Linux kernel. You're talking to a program called the **shell**. The shell is a command interpreter — it reads the text you type, figures out what you mean, and tells the operating system to do it.

The most common shell on Linux is **Bash** (Bourne Again SHell). It's the default on Ubuntu and most other major distributions. There are others — `zsh`, `fish`, `dash` — but Bash is what you'll encounter almost everywhere, so that's what this guide uses.

The shell works in a simple loop called the **REPL**:

1. **Read** — waits for you to type a command and press Enter
2. **Evaluate** — parses what you typed and figures out what to run
3. **Print** — displays the output (or an error)
4. **Loop** — goes back to waiting

Understanding this loop is the mental model behind everything in this chapter. Every command you type is one iteration of that cycle.

2.2 Anatomy of a Command

Before running any command, it helps to understand how commands are structured. Almost every Linux command follows the same pattern:

```
command [options] [arguments]
```

syntax

Part	What It Is	Example
command	The program to run	<code>ls</code>
options	Flags that change behaviour (- or --)	<code>-l</code> or <code>--all</code>
arguments	The target the command acts on	<code>/home/user/documents</code>

A concrete example:

```
$ ls -la /home/ashik
```

bash

Breaking that down:

- `ls` — the command (list directory contents)
- `-l` — option: use long format (show permissions, size, date)
- `-a` — option: show hidden files (those starting with a dot)
- `/home/ashik` — argument: the directory to list

TIP

Short options (`-l`, `-a`) can usually be combined: `-la` or `-al` both work. Long options (`--all`) cannot be combined and are typed in full.

2.3 Where Are You? — `pwd`

The very first thing to know in any terminal session is *where you are* in the filesystem. The command `pwd` (**p**rint **w**orking **d**irectory) answers that question instantly.

```
$ pwd
/home/ashik
```

bash

That output tells you you're currently inside `/home/ashik` — your home directory. Think of it like asking "where am I?" in a building, and the answer being "third floor, room 12."

`pwd` takes no options and no arguments. It just tells you your current location. Run it any time you feel lost.

2.4 What's Here? — `ls`

`ls` lists the contents of a directory — files, folders, and links. It's one of the commands you'll run hundreds of times a day.

Basic usage

```
$ ls
Desktop  Documents  Downloads  Music  Pictures  Videos
```

bash

The most useful `ls` options

Option	Meaning	Example Output Change
-l	Long format: permissions, size, date, owner	Detailed list per line
-a	Show hidden files (names starting with .)	Includes .bashrc, .ssh, etc.
-h	Human-readable sizes (KB, MB, GB)	Shows '4.2M' instead of '4321024'
-t	Sort by modification time, newest first	Most recently changed at top
-r	Reverse the sort order	Oldest or Z-A at top
--color	Colour-code output by file type	Dirs blue, executables green

Most common real-world usage:

```
$ ls -lah
total 64K
drwxr-xr-x  2 ashik ashik 4.0K Jun 10 09:21 .
drwxr-xr-x 18 ashik ashik 4.0K Jun 10 09:00 ..
-rw-r--r--  1 ashik ashik 3.9K Jun 10 09:21 .bashrc
-rw-r--r--  1 ashik ashik  220 Jun 10 09:00 .bash_logout
drwxr-xr-x  2 ashik ashik 4.0K Jun 10 09:15 Documents
-rw-r--r--  1 ashik ashik  14K Jun 10 09:18 notes.txt
```

bash

Reading the long format output:

Column	Meaning	Example
Permissions	Who can read/write/execute	drwxr-xr-x
Links	Hard link count	2
Owner	User who owns the file	ashik
Group	Group that owns the file	ashik
Size	File size (-h makes it readable)	4.0K
Date	Last modification time	Jun 10 09:21
Name	File or directory name	Documents

2.5 Moving Around — cd

cd (change **d**irectory) moves you to a different location in the filesystem. It's the equivalent of double-clicking a folder in a file manager.

Absolute vs. Relative Paths

There are two ways to specify a location in Linux, and understanding both is essential:

- **Absolute path** — starts from the root /, always works regardless of where you currently are. Example: /home/ashik/Documents
- **Relative path** — starts from your *current* directory. Example: if you're in /home/ashik, then Documents (no leading slash) refers to /home/ashik/Documents

Essential cd patterns

```
$ cd /home/ashik/Documents    # absolute path
$ cd Documents                # relative path (from home)
$ cd ..                       # go up one level (parent dir)
$ cd ../../..                 # go up two levels
$ cd ~                        # go to your home directory
$ cd -                        # go back to previous directory
$ cd                          # also goes to home (no argument)
```

bash

Special path symbols

Symbol	Means	Example
~	Your home directory	cd ~/Downloads
.	Current directory	ls . (same as ls)
..	Parent directory	cd ..
-	Previous directory you were in	cd -
/	Root of the entire filesystem	cd /

TIP

After every cd, it's good habit to run pwd to confirm where you ended up, then ls to see what's there. After a while this becomes instinct.

2.6 Getting Help: man, --help, and whatis

One of the most important skills in Linux isn't memorizing commands — it's knowing how to look things up. Linux comes with extensive built-in documentation.

The man command (manual pages)

`man` opens the full manual page for any command. Manual pages are the authoritative reference for every detail of a command's behaviour.

```
$ man ls
```

bash

```
# Opens the ls manual page. Navigate with:
#   Arrow keys or j/k   scroll line by line
#   Space / b          scroll page down / up
#   /keyword            search for a word
#   n                   jump to next search match
#   q                   quit
```

Quick help with `--help`

Almost every command accepts `--help` (or `-h`) as an option. This prints a short summary of the command's options — faster than a full man page when you just need a quick reminder.

```
$ ls --help
$ cd --help
```

bash

One-line descriptions with `whatis`

`whatis` gives you a single-line description of any command — useful when you encounter a command you've never seen before.

```
$ whatis ls
ls (1)          - list directory contents

$ whatis pwd
pwd (1)         - print name of current/working directory
```

bash

2.7 Command History and Shortcuts

The terminal remembers the commands you've typed. This history is stored in a file called `~/.bash_history` and is loaded every time you open a new terminal.

Navigating history

Key / Command	Action
Up arrow	Go to previous command
Down arrow	Go to next command

Key / Command	Action
Ctrl + R	Reverse search: type part of a command to find it
history	Print the last 500 commands with line numbers
! <u>42</u>	Re-run command number 42 from history
!!	Re-run the very last command
!!s	Re-run the most recent command starting with 'ls'

Keyboard shortcuts that save time

Shortcut	Action
Ctrl + A	Jump to start of line
Ctrl + E	Jump to end of line
Ctrl + U	Delete from cursor to start of line
Ctrl + K	Delete from cursor to end of line
Ctrl + W	Delete the word before the cursor
Ctrl + L	Clear the screen (same as clear command)
Ctrl + C	Cancel (kill) the currently running command
Ctrl + Z	Suspend a running command (send to background)
Alt + F / Alt + B	Move forward / backward one word at a time

TIP

Ctrl + R is arguably the most useful shortcut in Bash. Press it, start typing part of a previous command, and it finds the most recent match. Press Ctrl + R again to cycle through older matches.

2.8 Tab Completion: Your Best Friend

Tab completion is one of the most powerful features of the Bash shell, and new Linux users consistently say it was the thing that most changed how they work once they discovered it.

When you press the **Tab** key, Bash tries to complete what you're typing — whether that's a command name, a filename, a directory, or even an option flag.

How it works

```
$ cd Doc<Tab>
# Bash completes to:
$ cd Documents/

$ ls /home/as<Tab>
# Bash completes to:
$ ls /home/ashik/

$ sys<Tab><Tab>
# Bash shows all commands starting with 'sys':
sysctl    systemctl    systemd-analyze    syslog
```

- **Single Tab** — completes if there's only one possibility
- **Double Tab** — shows all possibilities when there are multiple matches
- Works for **commands**, **filenames**, **directory paths**, and even **usernames** (after ~)
- If nothing happens on the first Tab, it means no match was found or there are too many matches to auto-complete

TIP

Make a habit of using Tab constantly. It prevents typos, saves time, and also confirms that the file or command you're about to use actually exists before you run anything.

2.9 Clearing the Screen and Exiting

Two simple but essential things: keeping your terminal tidy, and knowing how to leave.

Clearing the screen

```
$ clear          # clears all visible output (history stays)
$ Ctrl + L      # keyboard shortcut for the same thing
$ reset         # fully resets the terminal (useful if it gets corrupted)
```

Note: `clear` only clears what's *visible*. Your command history and any background processes are not affected. You can still scroll up to see previous output in most terminal emulators.

Exiting the terminal

```
$ exit          # closes the current shell session
$ logout        # same as exit, works in login shells
$ Ctrl + D      # sends EOF signal – same effect as exit
```

If you're connected to a remote server via SSH, `exit` or `Ctrl + D` closes the SSH session and returns you to your local machine — your local terminal stays open.

2.10 Essential First Commands: Quick Reference

Here's a consolidated reference of every command and shortcut covered in this chapter:

Command / Key	What It Does	Common Usage
<code>pwd</code>	Print current directory	<code>pwd</code>
<code>ls</code>	List directory contents	<code>ls -lah</code>
<code>ls -l</code>	Long format listing	<code>ls -l /etc</code>
<code>ls -a</code>	Show hidden files too	<code>ls -la</code>
<code>cd</code>	Change directory	<code>cd ~/Documents</code>
<code>cd ..</code>	Go up one level	<code>cd ../..</code>
<code>cd ~</code>	Go to home directory	<code>cd ~</code>
<code>cd -</code>	Go to previous directory	<code>cd -</code>
<code>man</code>	Open full manual page	<code>man ls</code>
<code>--help</code>	Short usage summary	<code>ls --help</code>
<code>whatis</code>	One-line description	<code>whatis pwd</code>
<code>history</code>	Show command history	<code>history tail -20</code>
<code>Ctrl + R</code>	Search command history	(interactive)
<code>Tab</code>	Auto-complete command or path	(press after partial input)
<code>Ctrl + C</code>	Cancel running command	(emergency stop)
<code>Ctrl + L</code>	Clear screen	(any time)
<code>clear</code>	Clear screen	<code>clear</code>
<code>exit</code>	Close the terminal / SSH session	<code>exit</code>

2.11 Chapter Summary

- ✓ The **shell** (Bash) is the program that reads, evaluates, and executes your commands in a continuous loop.
- ✓ Every command follows the pattern: `command [options] [arguments]`. Short options can be combined (-lah); long ones cannot.
- ✓ `pwd` tells you where you are. `ls` shows what's here. `cd` takes you somewhere else.
- ✓ **Absolute paths** start with `/` and always work. **Relative paths** are relative to your current location.
- ✓ Special symbols: `~` = home, `.` = here, `..` = parent, `-` = previous location.
- ✓ `man`, `--help`, and `what is` are your built-in documentation system. Use them constantly.
- ✓ **Tab completion** is the most important habit to build. Press Tab early and often to complete paths and avoid typos.
- ✓ **Command history** (Up arrow, Ctrl+R) means you rarely need to retype a long command.
- ✓ The keyboard shortcuts Ctrl+C, Ctrl+L, Ctrl+A, and Ctrl+E will save you time every single day.

What's Coming in Chapter 3

Now that you can navigate the filesystem, Chapter 3 puts that knowledge to work. You'll learn how to *manipulate* the filesystem itself:

- Create files (`touch`) and directories (`mkdir`)
- Copy files and directories with `cp`
- Move and rename with `mv`
- Delete safely with `rm` — and why `rm -rf` deserves your full attention
- View file contents with `cat`, `less`, and `head / tail`

Chapter 3 is where the terminal stops feeling like a curiosity and starts feeling like a tool.

Author

Ashik A

github.com/ashihh07

This guide was created, organized, refined, and designed by Ashik A using GitHub documentation, industry best practices, publicly available learning resources, and AI-assisted research.
